# ESE5023 Assignment 01

Huang Xiaoqi 12632722

本次作业完成过程中，我参考了 CSDN 平台上的相关编程资料，以帮助理解部分算法思路和 Python 语法。作业代码与报告均由本人独立完成。

## 1. Flowchart

这道题的难点在于流程分支是否完整。三个数虽然只有六种大小顺序，但漏掉任何一条路径都会使某类输入得到错误结果，心里要有清晰的逻辑。完成代码后，我用多组不同排列的数逐一测试，尽可能覆盖多种情况来验证代码。这个过程让我认识到，写条件语句时既要保证逻辑与流程图一致，也要通过测试覆盖每一个可能的分支。

## 2. Matrix multiplication

自己用三重循环实现矩阵乘法后，我对矩阵行列相乘的计算过程理解得更具体了。外层循环决定结果矩阵的位置，内层循环把 M1 第 i 行与 M2 第 j 列的对应元素相乘并累加，其中公共维度必须保持一致。题目限制只能使用循环、乘法和加法，迫使我把平时由 NumPy 自动完成的步骤逐项展开。完成后再用 np.dot 核对结果，也让我意识到库函数不仅能提高效率，还可以作为验证自写算法是否正确的参照。

## 3. Pascal triangle

在帕斯卡三角题中，我尝试了两种生成第 k 行的方法。第一种从上一行出发，将相邻元素相加，过程直观，能够清楚展示三角形如何逐行形成；第二种利用组合数之间的递推关系，根据当前元素直接计算下一个元素，在求第 100 行和第 200 行时更加简洁。比较两种写法后，我认识到同一个问题往往有多种正确实现，选择时要同时考虑可读性、中间数据量和运行效率。

## 4. Add or double

这道题最重要的收获是学会从目标值反向推理。如果从 1 正向选择加 1 或乘 2，需要在许多可能路径中搜索。从目标 x 反推时，偶数可以除以 2，奇数的最后一步不可能由翻倍得到，因此只能先减 1。这样就把搜索问题变成了一条不断缩小数值的确定过程。我先用递归表达这种关系，再改写为 while 循环，两种实现对 2 和 5 等测试值给出了相同答案。这个比较让我更直观地理解了递归与迭代的转换，也提醒我在写算法前先寻找问题是否存在更简单的逆向结构。

## 5. Dynamic programming

八个数字间隙各有连接、加号和减号三种选择，因此共有 24 种组合。暂时没有什么很好的解题思路，所以使用了穷举的笨办法。先生成所有可能的表达式，再逐一计算并筛选符合目标值的结果。这个方法比较直接，而且搜索规模不大，能够顺利完成任务。后续再跟老师同学交流思路。